

MODULE 22

Generative AI and Large Language Models

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Generative AI and Large Language Models

What is Generative AI?

Generative AI creates new content: text, images, code, audio.

Types of Generative Models:

GAN: Generative Adversarial Networks (images)

VAE: Variational Autoencoders (images)

Diffusion Models: Stable Diffusion, DALL-E (images)

Transformers/LLMs: GPT-4, Claude, Gemini (text)

Applications: content creation, code generation, design, synthetic data

Large Language Models

LLMs are transformer-based models trained on massive text data.

Key LLMs:

GPT-4 / ChatGPT: OpenAI

Gemini: Google DeepMind

Claude: Anthropic

Llama 3: Meta (open source)

Mistral: open source

Capabilities:

Text generation, summarisation, translation,
code generation, question answering, reasoning

Using LLMs with Python

```
from openai import OpenAI

client = OpenAI(api_key='your-api-key')

# Chat completion
response = client.chat.completions.create(
    model='gpt-4',
    messages=[
        {'role': 'system', 'content': 'You are a data science expert.'},
        {'role': 'user', 'content': 'Explain overfitting in simple terms.'}
    ],
    max_tokens=500, temperature=0.7
)
print(response.choices[0].message.content)
```

```
# Embeddings
embedding = client.embeddings.create(
input='machine learning',
model='text-embedding-3-small'
)
```

RAG - Retrieval Augmented Generation

RAG = LLM + your own documents for accurate answers.

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI
from langchain.document_loaders import PyPDFLoader

# Load documents
loader = PyPDFLoader('document.pdf')
docs = loader.load_and_split()

# Create vector store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(docs, embeddings)

# Create QA chain
qa = RetrievalQA.from_chain_type(llm=OpenAI(), chain_type='stuff',
retriever=vectorstore.as_retriever())
answer = qa.run('What is the main topic?')
```

Fine-tuning LLMs

Fine-tuning adapts a pretrained LLM to your specific task.

```
from transformers import AutoModelForCausalLM, AutoTokenizer, Trainer,
TrainingArguments
from peft import LoraConfig, get_peft_model

# Load model
model = AutoModelForCausalLM.from_pretrained('meta-llama/Llama-2-7b-hf')
tokenizer = AutoTokenizer.from_pretrained('meta-llama/Llama-2-7b-hf')

# Apply LoRA (Low-Rank Adaptation) - efficient fine-tuning
lora_config = LoraConfig(r=16, lora_alpha=32, target_modules=['q_proj', 'v_proj'])
model = get_peft_model(model, lora_config)

# Training
training_args = TrainingArguments(output_dir='./results', num_train_epochs=3)
trainer = Trainer(model=model, args=training_args, train_dataset=dataset)
trainer.train()
```